SUBMISSION DATE
DECEMBER 19,2025

المدرسة الوطنية للعلوم التطبيقية - القنيطرة
National School of Applied Sciences - Kenitra

PROJECT REPORT

RETAIL SUPPLY CHAIN SALES

Team Members

- BENBOUAZZA NOHAYLA
- BOUF IMANE
- BELGHAZI YOUSSRA
- ED-DABARH KARIMA
- MOUDNI ZAKARIA
- ELHANSALY ABDULBARY

MODULE NAME
COMPETENCE NUMERIQUE

POFESSOR
DR.AJANA MEHDIA

Introduction

Analyzing retail sales and supply chain performance is essential for understanding market dynamics, customer behavior, and operational efficiency. This report examines a structured dataset focused on retail transactions and logistics, offering insights into commercial and supply chain activities across various regions and customer segments.

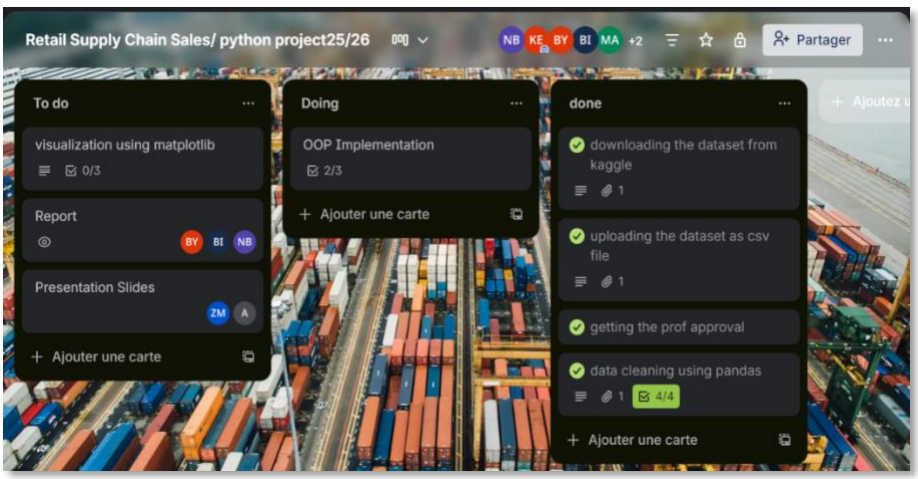
The dataset includes detailed attributes such as **Order ID, Order Date, Ship Date, Ship Mode, Customer ID, Customer Name, Segment, Country, City, Product Category, Sub-Category, and Product Name.** It also provides key financial metrics including **Sales, Quantity, Discount, and Profit.**

This data enables comprehensive analysis of sales trends, customer segmentation, discount impact on profitability, and logistics performance. It serves as a robust foundation for data science applications such as visualization, predictive modeling, and behavioral clustering.

Methods

In order to explore and present the dataset effectively, we designed a systemat workflow built around the following steps:

- ✓ **Workflow Structuring :**
  Our first action was to set up a Trello board involving all team members, providing a structured platform for collaboration and workflow organization.
  https://trello.com/invite/b/6932eb2e5ee69e0e8a0b0a34/ATTI9e7a644fc4a6b256ad84402e324b684bD5697BED/retail-supply-chain-sales-python-project25-26



✓ **Dataset Selection and Preparation:**

The second step consisted of acquiring the Retail Supply Chain Sales dataset from **Kaggle**. In light of platform limitations and project requirements, the dataset was refined by removing non-essential records to ensure compliance with the row limit while preserving the core attributes necessary for analysis.

✓ **Data Loading:**

```
import pandas as pd
# 1. LOAD DATA
df = pd.read_csv("DATA_SET.csv")
```

The dataset "*DATA_SET.csv*" was imported using Pandas.

✓ **Dataset Inspection:**

```
# 2. INSPECT THE DATASET
print("Dataset Description:")
print(df.describe(include="all"))
print("First 10 rows:")
print(df.head(10))
print("Last 10 rows:")
print(df.tail(10))
print("Column Names and Data Types:")
print(df.dtypes)
print("Missing Values Per Column:")
print(df.isnull().sum())
```

A preliminary inspection was conducted to understand the dataset's structure and content:

Descriptive statistics were generated for all columns.

The **first 10** and **last 10 rows** were displayed to gain an overview of the records.

Column names and data types were examined to identify variable formats.

Missing values per column were calculated to assess data completeness.

✓ **Handle Incorrect Data Types**

```
# 3. HANDLE WRONG DATA TYPE
# Convert Quantity to numeric (invalid values become NaN)
df["Quantity"] = pd.to_numeric(df["Quantity"], errors="coerce")
```

Converts the `"Quantity"` column to **numeric format**.
If a value can't be converted it becomes **NaN.**

✓ **Summary Statistics:**

```
# 4. SUMMARY STATISTICS
# Quantity sum
quantity_sum = df["Quantity"].sum()
# Sales MEAN, MAX, MIN
sales_mean = df["Sales"].mean()
sales_max = df["Sales"].max()
sales_min = df["Sales"].min()
print("Summary Statistics:")
print(f"Total Quantity (sum): {quantity_sum}")
print(f"Sales Mean: {sales_mean}")
print(f"Sales Max: {sales_max}")
print(f"Sales Min: {sales_min}")
```

Key statistical measures were computed to summarize the dataset:

Total Quantity: The sum of all quantities sold.

Sales Metrics: Mean, maximum, and minimum values of sales were calculated to evaluate distribution and range.

✓ **Handling Missing Data:**

```
# 5. HANDLE MISSING DATA
# Removing rows that contain any missing values
df_dropna = df.dropna()
# Filling missing values (example: fill numeric columns with 0)
df_filled = df.fillna(0)
print("After Dropping Rows with Missing Values:")
print(df_dropna.isna().sum())
print("After Filling Missing Values with 0:")
print(df_filled.isna().sum())
```

Two approaches were applied to address missing values:

Row removal: Eliminating records containing any missing values.

Imputation: Filling missing values with zero for numeric columns.

This ensured the dataset was clean and ready for further analysis.

✓ **Data Filtering:**

```
# 6. FILTERING DATA
# Quantity > 10
quantity_10 = df[df["Quantity"] > 10]
print("Quantity > 10:")
print(quantity_10)
```

A filtering operation was performed to extract records where **Quantity > 10**, allowing focused analysis on larger transactions.

✓ **Grouped Analysis:**

```
# 7. GROUP BY DATA
# Mean Sales for each City
each_city_average_sales = df.groupby("City")["Sales"].mean()
# Total Profit for each City
each_city_profit_sum= df.groupby("City")["Profit"].sum()
print("Each city average sales:")
print(each_city_average_sales)
print("Each city profit sum:")
print(each_city_profit_sum)
```

Aggregated statistics were generated by grouping data by **City**:

Average Sales per City: To compare sales performance acrossregions.

Total Profit per City: To evaluate profitability distribution Geographically.

✓ **Visualization Steps:**

We used Python's matplotlib and pandas libraries to create visualizations that reveal trends in the dataset:

- **First visualization : Top 10 Cities with Highest Sales**

```
import pandas as pd
import matplotlib.pyplot as plt
# 1. LOAD DATASET
df = pd.read_csv("DATA_SET.csv")
# ======== BAR CHART : TOP CITIES BY TOTAL SALES =========
# 1. Calculate total sales for each city
city_sales = df.groupby("City")["Sales"].sum()
# 2. Select the Top 10 cities
top_cities = city_sales.sort_values(ascending=False).head(10)
# 3. Create bar chart
plt.figure(figsize=(11, 6))
plt.bar(top_cities.index, top_cities.values, color="cornflowerblue")
# 4. Add title and labels
plt.title("Top 10 Cities by Total Sales", fontsize=14, fontweight="bold")
plt.xlabel("City")
plt.ylabel("Total Sales")
# 5. Improve readability
plt.xticks(rotation=45)
plt.grid(axis="y", linestyle="--", alpha=0.6)
# 6. Show plot
plt.show()
```

Steps :

1. Grouping the dataset by City and calculating Total sales for each city.
2. Get the Top 10 cities with the highest sales.

3. Create a figure and plot a blue bar chart of total sales for the top 10 cities.
4. Add a title and label the x-axis as “City” and the y-axis as “Total Sales.”
5. Rotate the x-axis labels by 45 degrees and add horizontal dashed grid lines for readability.
6. Display the chart on the screen.

- **Second visualization : Sales Trend Over Time**

```
# ======== LINE PLOT : SALES TREND OVER TIME =====
# 1. Convert Order Date to datetime format
df["Order Date"] = pd.to_datetime(df["Order Date"], errors="coerce")
# 2. Remove rows with invalid dates
df = df.dropna(subset=["Order Date"])
# 3. Calculate total sales per date
daily_sales = df.groupby("Order Date")["Sales"].sum()
# 4. Create line plot
plt.figure(figsize=(12, 5))
plt.plot(
    daily_sales.index,
    daily_sales.values,
    color="green",
    linestyle="-",
    linewidth=2,
    marker="o",
    markersize=4)
# 5. Add title and labels
plt.title("Sales Trend Over Time", fontsize=14, fontweight="bold")
plt.xlabel("Order Date")
plt.ylabel("Total Sales")
# 6. Improve readability
plt.xticks(rotation=45)
plt.grid(True, linestyle="--", alpha=0.5)
# 7. Show plot
```

Steps :

1. Converts the `"Order Date"` column to proper **datetime objects**.If a date is invalid it's replaced with **NaT** (Not a Time).
2. Drops rows where `"Order Date"` is missing or invalid.
3. Groups the data by date and sums the `"Sales"` for each day.
4. Plots a green line with circular markers to show daily sales.
5. Adds a bold title and labels for both axes.
6. Rotates x-axis labels to avoid overlap.Adds a light dashed grid to help visualize trends.

7. Displays the final chart.

- **Third visualization : Discount vs Sales**

Steps :

```
# ======== SCATTER PLOT : DISCOUNT VS SALES ================
# 1. Create scatter plot
plt.figure(figsize=(8, 6))
plt.scatter(
    df["Discount"],
    df["Sales"],
    color="purple",
    alpha=0.5)
# 2. Add title and labels
plt.title("Relationship Between Discount and Sales", fontsize=14, fontweight="bold")
plt.xlabel("Discount")
plt.ylabel("Sales")
# 3. Add grid
plt.grid(True, linestyle="--", alpha=0.5)
# 4. Show plot
plt.show()
```

1. Plots each data point where:**x-axis** = Discount,**y-axis** = SalesUses **purple** dots with `alpha=0.5` for semi-transparency, which helps when points overlap.
2. Adds a bold title and labels for both axes to clarify what the chart shows.
3. Adds a light dashed grid to help interpret the position of points.
4. Displays the final scatter plot.

- **fourth visualization : sales distribution by customer segment**

Steps :

```
# ======== PIE CHART : SALES BY CUSTOMER SEGMENT ===========
# 1. Calculate total sales per customer segment
segment_sales = df.groupby("Segment")["Sales"].sum()
# 2. Define colors and explode effect
colors = ["gold", "lightskyblue", "lightcoral"]
explode = [0.05] * len(segment_sales)
# 3. Create pie chart
plt.figure(figsize=(7, 7))
plt.pie(
    segment_sales,
    labels=segment_sales.index,
    autopct="%1.1f%%",
    startangle=90,
    colors=colors,
    explode=explode,
    shadow=True,
    textprops={"fontsize": 11})
# 4. Add title and legend
plt.title("Sales Distribution by Customer Segment", fontsize=14, fontweight="bold")
plt.legend(title="Customer Segment")
# 5. Show plot
plt.show()
```

1. Groups the data by `"Segment"`. Sums the `"Sales"` for each segment to get total sales per category.
2. Sets custom colors for each slice of the pie. Applies a slight **explode effect** (pulls slices outward) for visual emphasis.

3. Plots the pie chart: labels: segment names, autopct: shows percentage values with 1 decimal, startangle=90: rotates the chart for better alignment, shadow=True: adds depth, textprops: sets font size for labels.

4. Adds a bold title and a legend to clarify the meaning of each slice.

✓ **OOP Implementation:**

To structure a data analysis workflow in Python we create a class with methods like load_data() to import the dataset, analyze_data() to compute basic statistics, and plot_data() to generate visualizations. This approach organizes code, makes it reusable, and simplifies extending analysis functionality.

- **Creating a Python class with the following simple methods:**

load_data() for importing the dataset:

```
import pandas as pd
import matplotlib.pyplot as plt
class Analyzes:
    def __init__(self, file_name):
        self.file_name = file_name
        self.df = None   # we will load the dataframe here later
    def load_data(self):
        """Loads the CSV dataset."""
        try:
            self.df = pd.read_csv(self.file_name)
            print("Dataset loaded successfully.")
        except Exception as e:
            print("Error loading dataset:", e)
```

The load_data method reads a CSV file using pandas with semicolon separators and comma decimals, storing it in self.df. It prints a success message if loading works or an error message if it fails.

analyze_data() for generating basic statistics:

```python
def analyze_data(self):
    if self.df is None: #to make sure that the data is loaded
        print("Data not loaded yet!")
        return
    print("FIRST ROWS:")
    print(self.df.head())
    print("INFO:")
    print(self.df.info())
    print("DESCRIBE:")
    print(self.df.describe(include='all'))
    print("MISSING VALUES:")
    print(self.df.isna().sum())
```

The analyze_data method gives a quick dataset overview by displaying the first rows, dataset info (columns, types, non-null counts), summary statistics, and missing value counts to assess structure and completeness.

plot_data() for creating visualizations :

```python
def plot_data(self):
    """Create basic visualizations using Matplotlib."""
    if self.df is None:
        print("Data not loaded yet!")
        return

    # 1. BAR CHART: Total Sales by Category
    if 'Category' in self.df.columns:
        sales_by_category = self.df.groupby('Category')['Sales'].sum().sort_values(ascending=False)
        plt.figure(figsize=(10, 6))
        plt.bar(sales_by_category.index, sales_by_category.values, color='cornflowerblue')
        plt.title('Total Sales by Product Category', fontsize=14, fontweight='bold')
        plt.xlabel('Product Category', fontsize=12)
        plt.ylabel('Total Sales ($)', fontsize=12)
        plt.xticks(rotation=45)
        plt.grid(axis='y', linestyle='--', alpha=0.4)
        plt.show()
```

```
        # 2. SCATTER PLOT: Profit vs Discount
        if 'Discount' in self.df.columns and 'Profit' in self.df.columns:
            plt.figure(figsize=(8, 6))
            plt.scatter(
                self.df['Discount'],
                self.df['Profit'],
                color='purple',
                alpha=0.6,
                edgecolors='black',
                s=60)
            plt.title('Profit vs Discount', fontsize=14, fontweight='bold')
            plt.xlabel('Discount (%)', fontsize=12)
            plt.ylabel('Profit ($)', fontsize=12)
            plt.grid(True, linestyle='--', alpha=0.4)
            plt.show()

# USING THE CLASS
analysis = Analyzes("DATA_SET.csv")
analysis.load_data()
analysis.analyze_data()
analysis.plot_data()
```

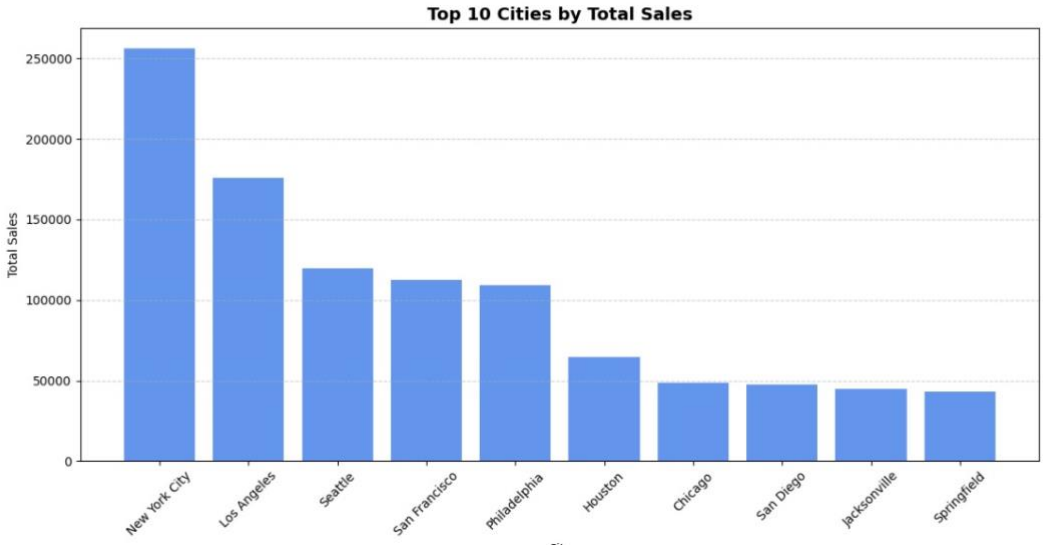
The plot_data method creates visualizations from self.df, including a bar chart of total sales by product category. It also produces a scatter plot of profit versus discount to highlight relationships between pricing and profitability.

RESULTS

Our analysis provided a comprehensive understanding of the dataset, highlighting trends related to retail supply chain sales, category distributions, and performance dynamics. Key findings and their corresponding visualizations are presented below:

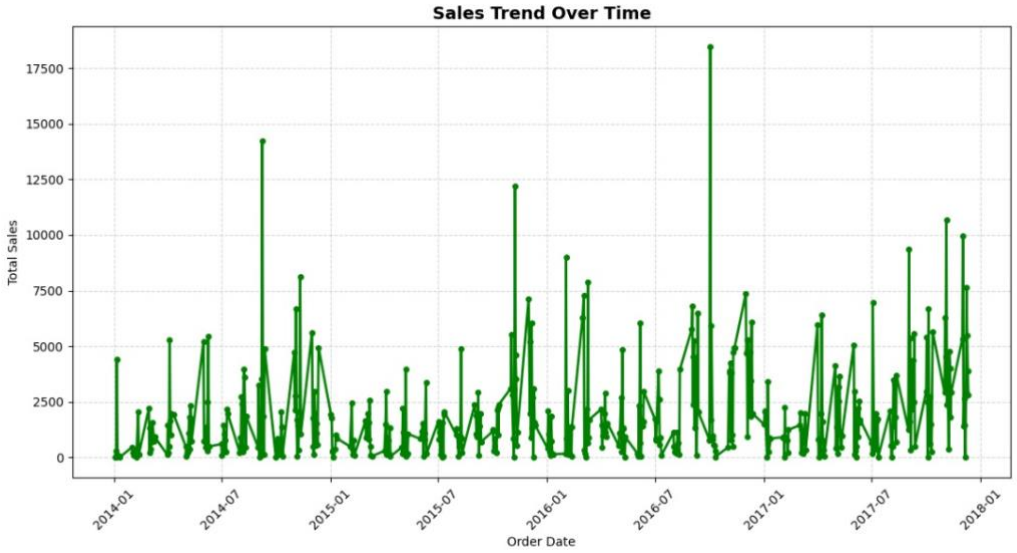
Top 10 Cities with Highest Sales:

The bar chart reveals that New York City leads retail supply chain sales by a significant margin, followed by Los Angeles and Seattle. These top-performing cities indicate strong market demand and distribution efficiency. Dallas, while still in the top 10, shows the lowest sales among them, suggesting potential for growth or reevaluation of supply strategies.



Sales Trend Over Time:

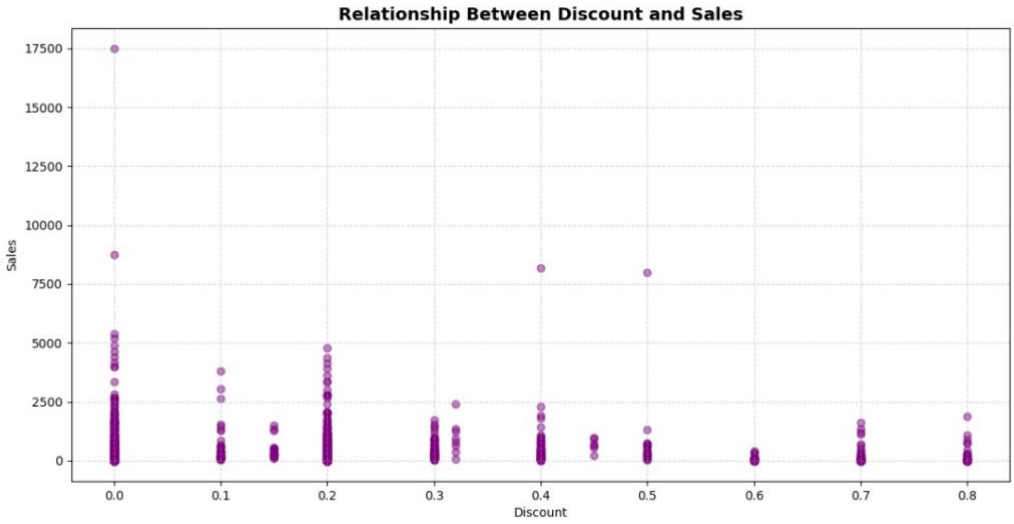
The line graph illustrates fluctuations in retail supply chain sales between 2014 and early 2018. Several sharp peaks suggest periods of high demand or promotional activity, while dips may reflect seasonal slowdowns or logistical constraints. Overall, the trend shows dynamic sales behavior, useful for identifying strategic planning windows.



Discount vs Sales:

This scatter plot explores how different discount levels affect retail supply chain sales. The wide spread of data points suggests that higher discounts do not consistently lead to higher sales. Some strong sales occur at low to moderate

discount rates, indicating that other factors—such as product type or timing—may influence performance more than discount alone.

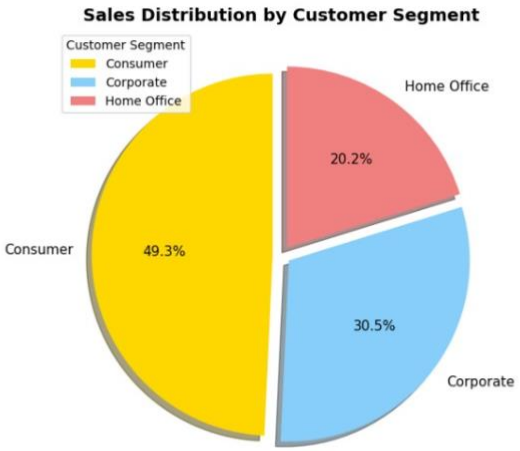


sales distribution by customer segment

Consumer segment contributes the largest share of sales, almost half at **49.3%**.

Corporate segment makes up about **30.5%**, a strong secondary contributor.

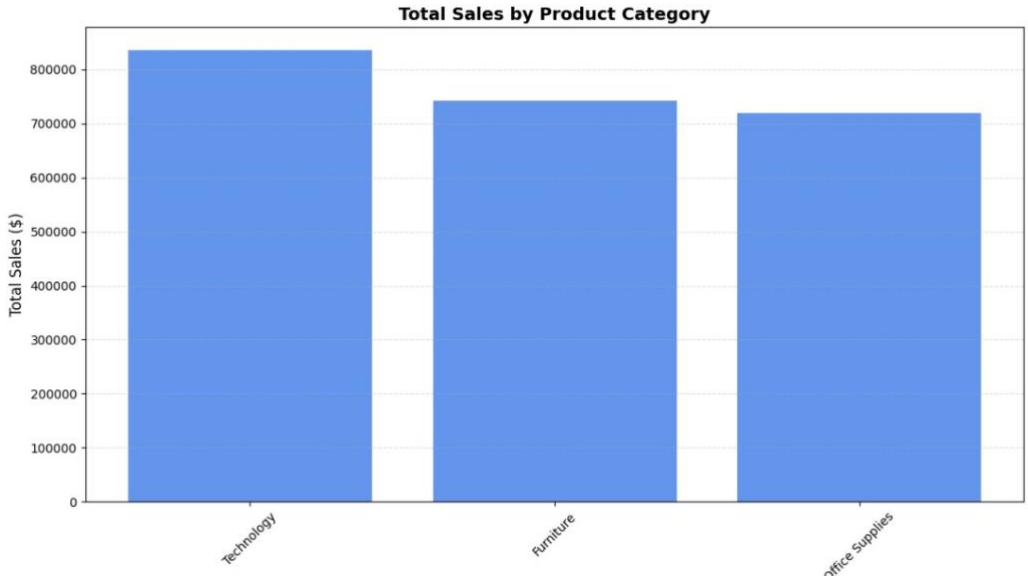
Home Office segment accounts for the smallest share at **20.2%.**



Total Sales by Product Category:

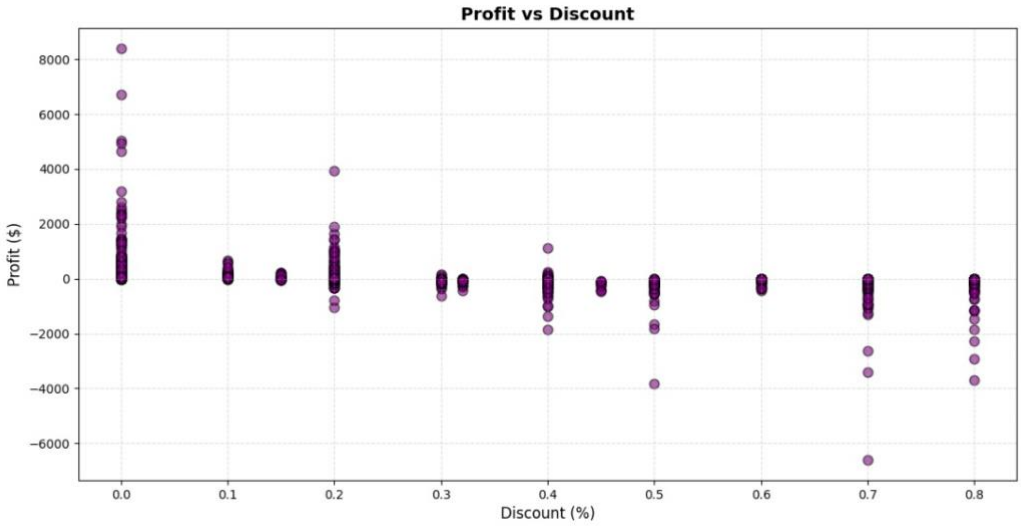
The bar chart compares total sales across three product categories in the retail supply chain. Technology leads with the highest sales, followed closely by Furniture, while Office Supplies ranks third. This distribution suggests that

investment and marketing efforts may be most effective when focused on technology-related products.



Profit Vs Discount:

The scatter plot highlights the relationship between discount levels and profit in the retail supply chain. It shows that higher discounts often lead to reduced or even negative profits, while lower discounts tend to maintain profitability. This suggests that aggressive discounting may harm margins and should be carefully evaluated within pricing strategies.



Challenges Faced

During the development of this project, several challenges were encountered. One of the first difficulties was related to data quality. The original dataset was already relatively clean, which limited opportunities to practice data cleaning techniques. To address this, missing and inconsistent values were intentionally introduced and then handled using different strategies such as row removal and numerical imputation, allowing us to apply and demonstrate data preprocessing methods effectively.

Another challenge involved encoding issues when running the code on different personal computers. In some environments, particularly on certain systems, the dataset could not be read correctly using UTF-8 encoding. This problem was resolved by switching to the Latin-1 encoding format, ensuring compatibility and successful data loading across devices.

Additionally, the team faced difficulties in choosing consistent visualization styles and page layouts for plots and results. Different preferences initially led to inconsistencies; however, through discussion and collaboration, the team agreed on a unified visualization approach to maintain clarity and coherence throughout the project.

Finally, time management was a significant challenge due to tight deadlines and academic workload. Despite the pressure, the team successfully organized tasks using Trello, distributed responsibilities efficiently, and met all project requirements within the given timeframe.

CONCLUSION

This project provided valuable insights into retail supply chain sales by analyzing product categories, regional performance, discount strategies, and profitability trends. The results demonstrated that technology products generate the highest sales, major cities dominate overall revenue, and excessive discounting can negatively impact profit margins.

Beyond the analytical findings, this project strengthened our practical skills in data analysis using Python. The use of Pandas enabled efficient data manipulation, Matplotlib facilitated clear and meaningful visualizations, and Object-Oriented Programming improved code organization and reusability. Moreover, effective teamwork and communication were essential in overcoming technical and time-related challenges.

Overall, the project enhanced both our technical expertise and collaborative abilities, providing a strong foundation for future data science and supply chain analysis projects.